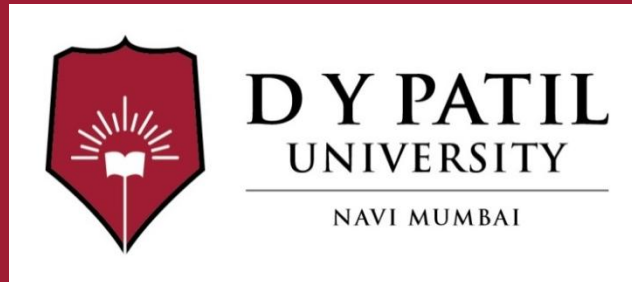


Subject Name :

Unit No: 4

Unit Name : Support Vector Machine

Faculty Name :



Unit No.: 4

Faculty Name : Dr. Ritu Jain

Index

Lecture No & Topic Name	Slide No
Support Vector Machines SVM Fundamentals: Margins, Support Vectors, Hard Margin, Soft margin (C parameter), Optimization	
Linear SVM	
Nonlinear SVM	
Nonlinear SVM: Kernel Trick, RBF	
Multiclass Classification (One-vs-One, One-vs-All)	



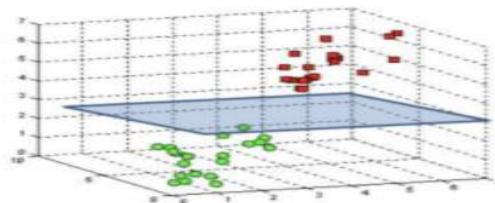
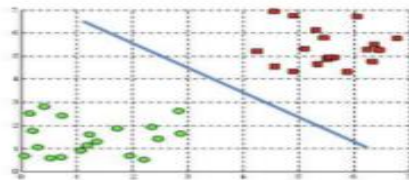
Unit No:1

Lecture No: 1



Hyperplane

- Hyperplanes are decision boundaries that help classify the data points.
- Data points falling on either side of the hyperplane can be attributed to different classes.
- Also, the dimension of the hyperplane depends upon the number of features.
- If the number of input features is 2, then the hyperplane is just a line.
- If the number of input features is 3, then the hyperplane becomes a two-dimensional plane.



Hyperplanes in 2D and 3D feature space



Linear Separability

- **Linear separability in machine learning refers to the ability to classify data points into two distinct classes using a single straight line (in 2D), plane (in 3D), or hyperplane (in n-dimensions).**
- If a straight boundary can perfectly separate two classes, the data is linearly separable; if not, it is nonlinearly separable.
- Two sets of data points are linearly separable if they can be separated by a linear decision surface (hyperplane), meaning all points of one class lie on one side of the surface, and all points of the other class lie on the other side.
- **Linear Models:** Algorithms such as Linear Regression, Logistic Regression, Naive Bayes, and Perceptrons, SVM with Linear Kernel work well on linearly separable data.



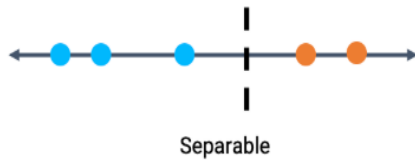
Non-linearly separable data

- Non-linearly separable data is described as data that cannot be separated using a simple straight line (requires a complex classifier).
- **Non-Linear Models:** When data is not linearly separable, following models are needed to create complex, non-linear boundaries.
 - Neural Networks with hidden layers,
 - Support Vector Machines (SVM) with kernels,
 - decision trees,
 - Random Forest, and
 - K-Nearest Neighbours (KNN)
- Non-linear classification refers to categorizing those instances that are not linearly separable.

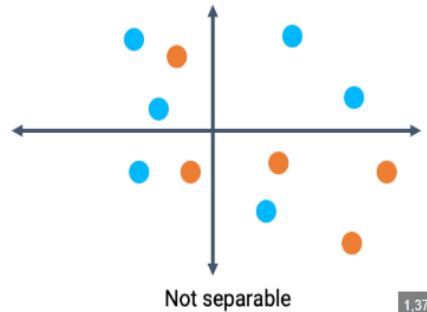
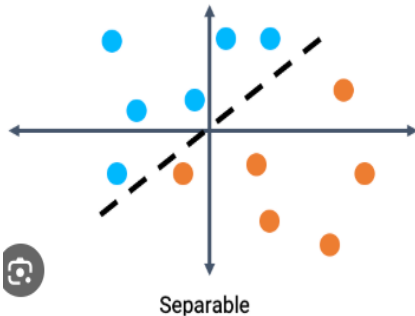


Examples of Linearly and Non Linearly Separable Data Set

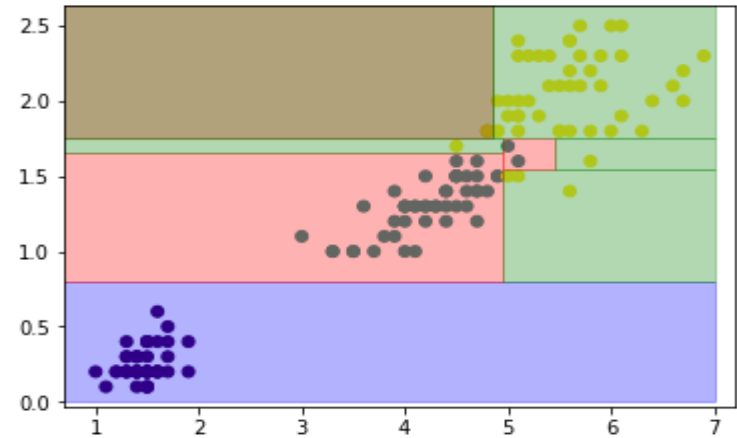
Dataset with one feature:



Dataset with two features:



ai

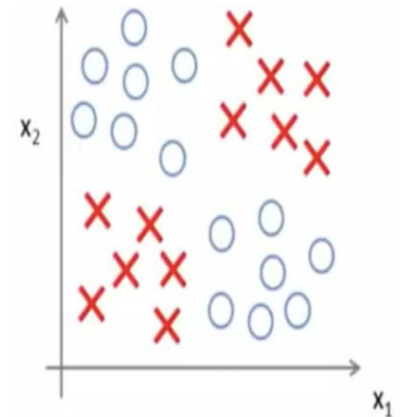
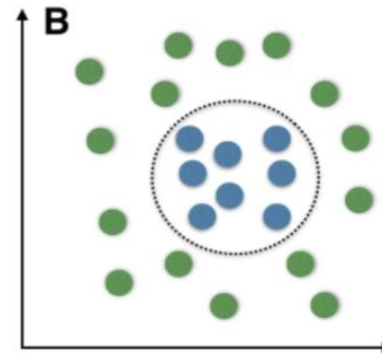
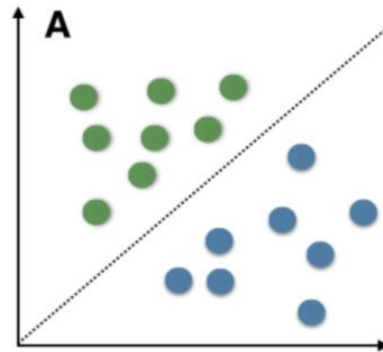
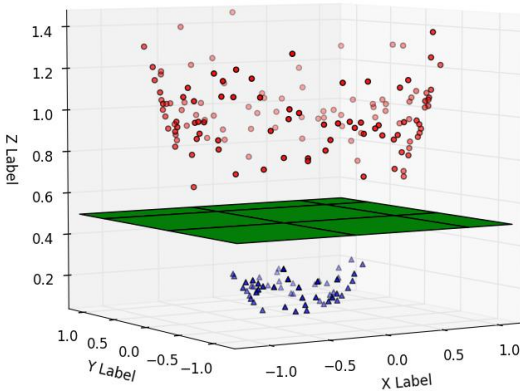


Non-Linearly Separable Data Set



How to Identify Linear Separability

- **Visual Inspection:** Plot the data in 2D or 3D to see if a straight line/plane can divide the groups.



A: Linearly Separable Data B: Non-Linearly Separable Data



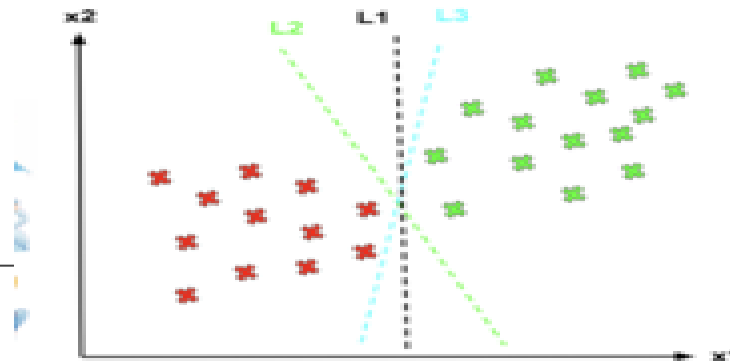
Introduction to Support Vector Machine (SVM)

- “Support Vector Machine” (SVM) is a supervised machine learning algorithm which can be used for both **classification or regression challenges**.
- However, it is **mostly used in classification problems**.
- In the SVM algorithm, we plot each data item as a point in n-dimensional space (where n is number of features you have) with the value of each feature being the value of a particular coordinate.
- Then, we perform classification by finding the hyper-plane that differentiates the two classes very well.



SVM

- Imagine the labelled training set are two classes of data points.
- To separate the two classes, there are so many possible options of hyperplanes that separate correctly. As shown in the graph below, we can achieve exactly the same result using different hyperplanes (L1, L2, L3).
- However, if we add new data points, the consequence of using various hyperplanes will be very different in terms of classifying new data point into the right group of class.



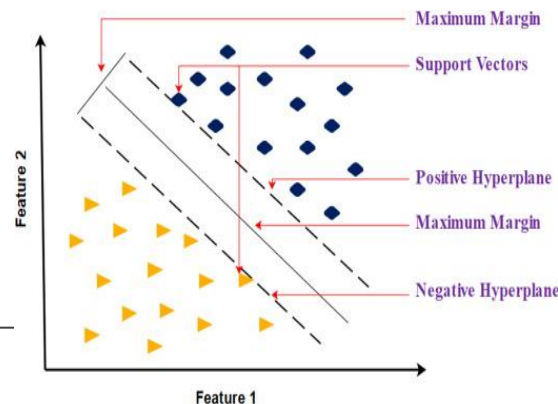
SVM

- ✓ A *Support Vector Machine* (SVM) is a very powerful and versatile Machine Learning model, capable of performing linear or nonlinear classification, regression, and even outlier detection. It is one of the most popular models in Machine Learning.
- ✓ SVMs are particularly well suited for classification of complex but small- or medium-sized datasets.
- ✓ You can think of an SVM classifier as fitting the widest possible margin (represented by the parallel dashed lines) between the classes.



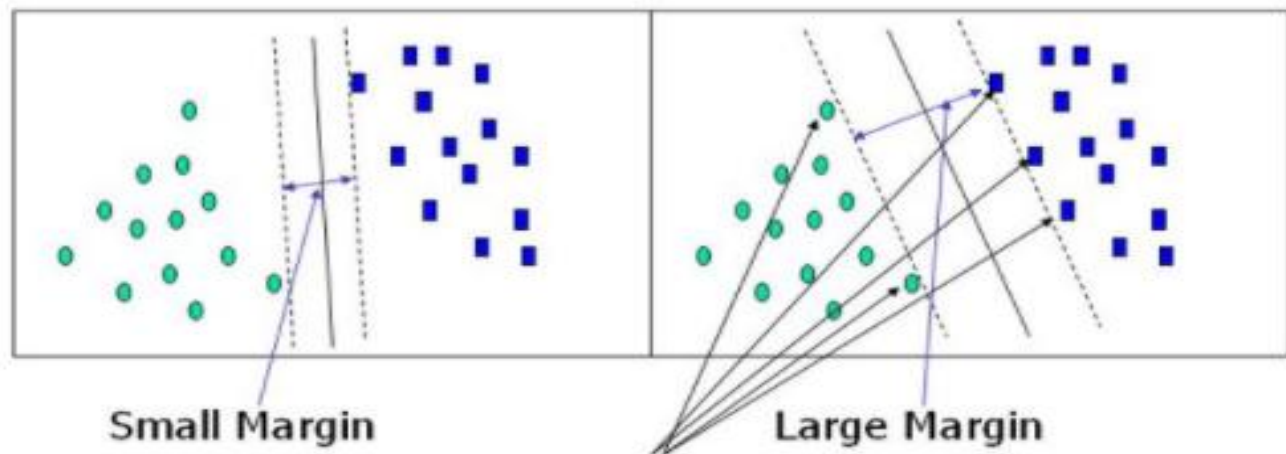
How can we decide a separating line for the classes? Which hyperplane shall we use?

- To separate the two classes of data points, there are many possible hyperplanes that could be chosen.
- ✓ • **Hyperplane**: A decision boundary separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.
- ✓ • **Margin** is the distance between the decision boundary (hyperplane) and the closest data points from each class, known as **support vectors**.
- ✓ • **Optimal Hyperplane**: Our objective is to find a plane that has the maximum margin, i.e the maximum distance between data points of both classes.



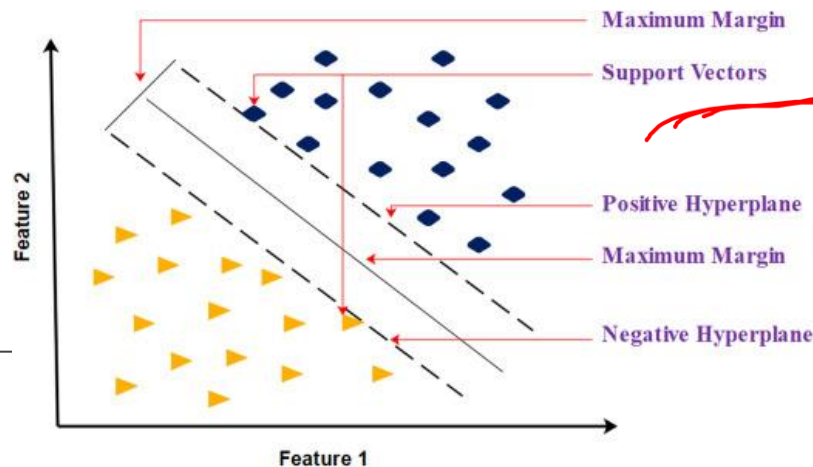
Main Goal of SVM

- The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data. Thus, it reduces the generalisation error
- Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence.
- The decision boundaries with smaller margins usually lead to overfitting.



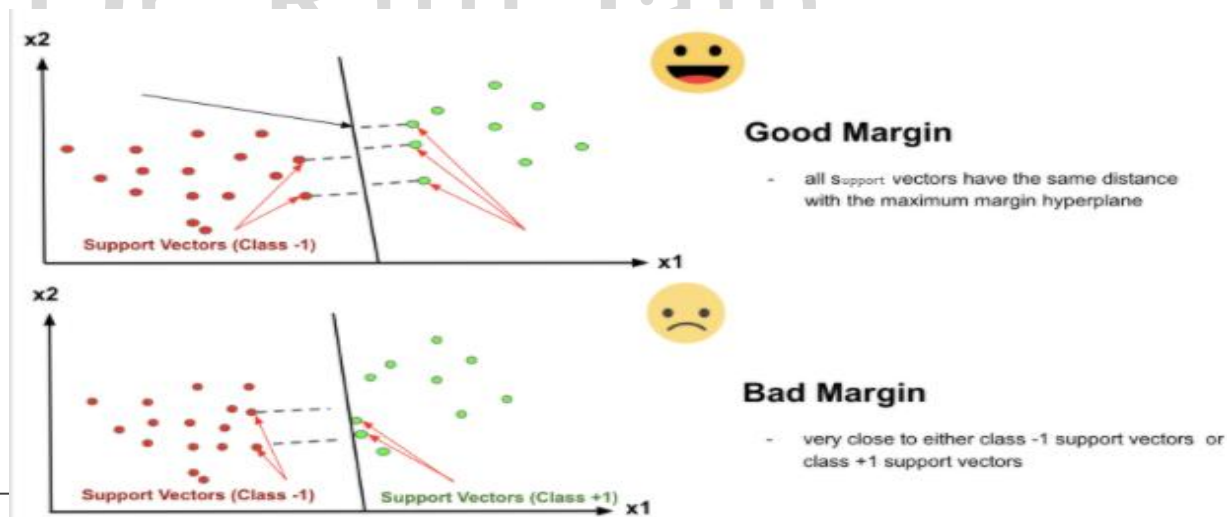
Support Vector

- **Support vectors** are data points that are closest to the hyperplane and influence the position and orientation of the hyperplane.
- Using these support vectors, we maximize the margin of the classifier.
- Deleting the support vectors will change the position of the hyperplane.
- These are the points that help us build our SVM.
- In other words, only support vectors points contributes to the result of the algorithm, and other points are not.
- If a data point is not a support vector, removing it has no effect on the model. On the other hand, deleting the support vectors will then change the position of the hyperplane.



Margin

- The distance of the support vectors from the hyperplane is called the **margin**, which is a separation of a line to the closest class points.
- We would like to choose a hyperplane that maximises the margin between classes. The graph below shows what good margin and bad margin are.



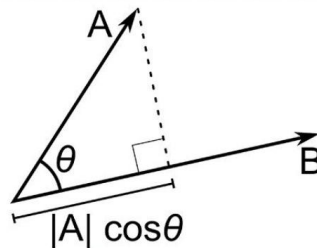
Large Margin Intuition

- In logistic regression, we take the output of the linear function and squash the value within the range of $[0,1]$ using the sigmoid function.
- If the squashed value is greater than a threshold value(0.5) we assign it a label 1, else we assign it a label 0.
- In SVM, we take the output of the linear function and if that output is greater than 1, we identify it with one class and if the output is less than -1, we identify it with another class.
- Since the threshold values are changed to 1 and -1 in SVM, we obtain the range of values $[-1,1]$ which acts as margin.



Mathematical Intuition: Support Vector Machine (SVM)

- ✓ **Vector:** A vector is a quantity that has magnitude as well as direction and just like numbers we can use mathematical operations such as addition, multiplication.
- The dot product can be defined as the projection of one vector along with another, multiply by the product of another vector.



Mathematically it can be written as:

$$\mathbf{A} \cdot \mathbf{B} = |\mathbf{A}| \cos\theta * |\mathbf{B}|$$

Where

$|\mathbf{A}| \cos\theta$ is the projection of A on B

And $|\mathbf{B}|$ is the magnitude of vector B

- ✓ Now in SVM we just need the projection of A not the magnitude of B.
- To just get the projection we can simply take the unit vector of B because it will be in the direction of B but its magnitude will be 1. Hence now the equation becomes:
- ✓ $\mathbf{A} \cdot \mathbf{B} = |\mathbf{A}| \cos\theta * \text{unit vector of B}$

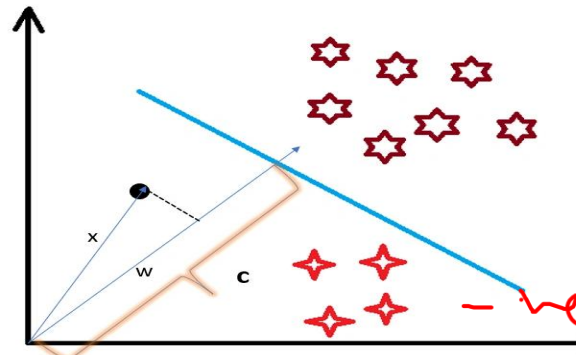
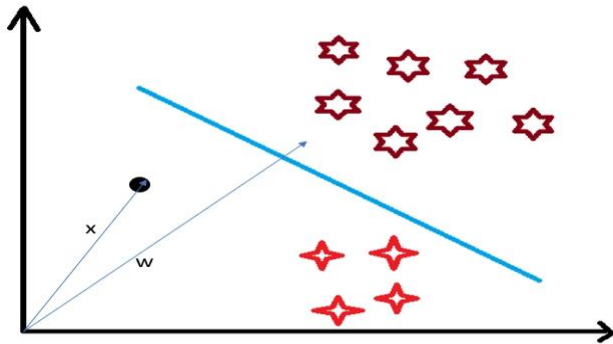


D Y PATIL
UNIVERSITY
NAVI MUMBAI

Mathematical Intuition: Support Vector Machine (SVM)

- **Use of Dot Product in SVM**

- Consider a random point X and we want to know whether it lies on the right side of the plane or the left side of the plane (positive or negative).
- To find this first we assume this point is a vector (X) and then we make a vector (w) which is perpendicular to the hyperplane. Let's say the distance of vector w from origin to decision boundary is ' c '. Now we take the projection of X vector on w .



+ve

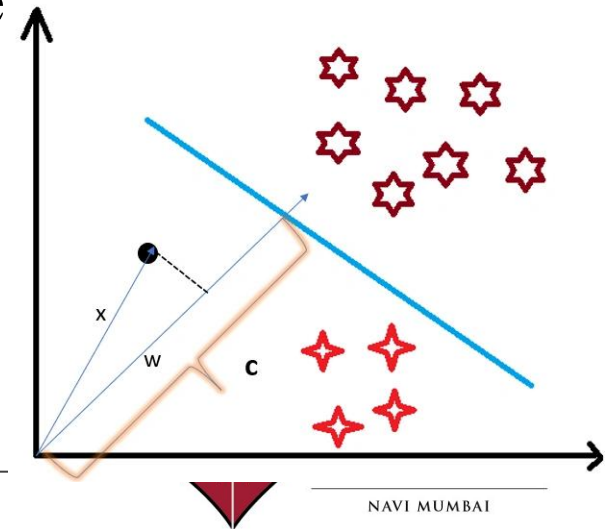
Mathematical Intuition: Support Vector Machine (SVM)

- We already know that projection of any vector or another vector is called dot-product.
- Hence, we take the dot product of x and w vectors. If the dot product is greater than ' c ' then we can say that the point lies on the right side.
- If the dot product is less than ' c ' then the point is on the left side and if the dot product is equal to ' c ' then the decision boundary.

$$\vec{X} \cdot \vec{w} = c \text{ (the point lies on the decision boundary)}$$

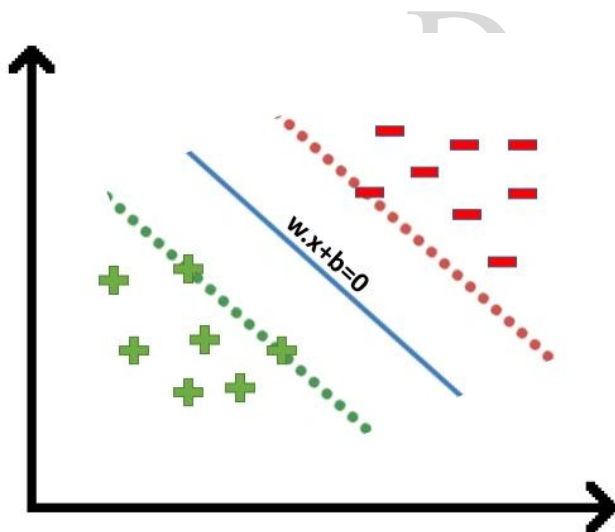
$$\vec{X} \cdot \vec{w} > c \text{ (positive samples)}$$

$$\vec{X} \cdot \vec{w} < c \text{ (negative samples)}$$



Margin in Support Vector Machine

- We all know the equation of a hyperplane is $w \cdot x + b = 0$ where
 - w is a vector normal to hyperplane and
 - b is an offset (how far the hyperplane is shifted from the origin).
- To classify a point as negative or positive we need to define a decision rule. We can define decision rule as:



$$\vec{X} \cdot \vec{w} - c \geq 0$$

putting $-c$ as b , we get

$$\vec{X} \cdot \vec{w} + b \geq 0$$

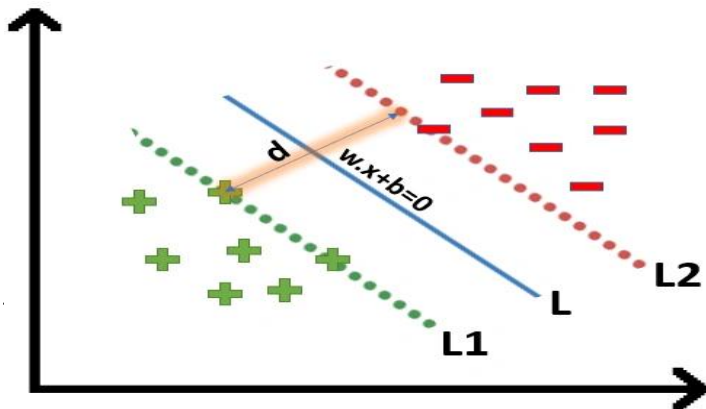
hence

$$y = \begin{cases} +1 & \text{if } \vec{X} \cdot \vec{w} + b \geq 0 \\ -1 & \text{if } \vec{X} \cdot \vec{w} + b < 0 \end{cases}$$



Mathematical Intuition: Support Vector Machine (SVM)

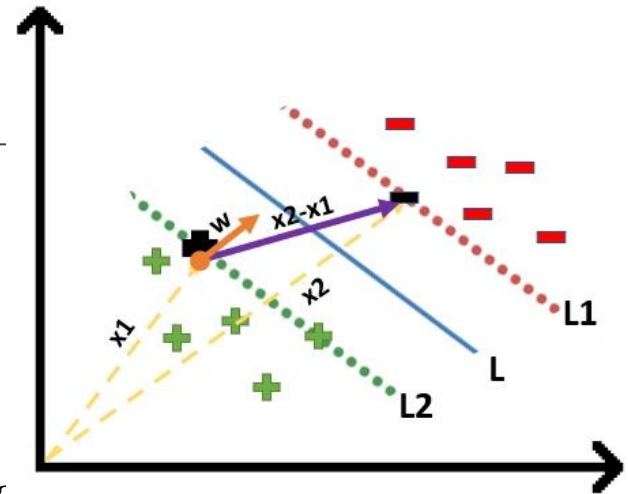
- If the value of $w \cdot x + b > 0$ then we can say it is a positive point otherwise it is a negative point. Now we need (w, b) such that the margin has a maximum distance. Let's say this distance is 'd'.
- To calculate 'd' we need the equation of L1 and L2. For this, we will take few assumptions that the equation of L1 is $w \cdot x + b = 1$ and for L2 it is $w \cdot x + b = -1$.
- We want our plane to have equal distance from both the classes that means L should pass through the center of L1 and L2 that's why we take magnitude equal.



Optimization Function and its Constraints

- We know that the aim of SVM is to maximize this margin that means distance (d).
- In order to get our optimization function, there are few constraints to consider.
- That constraint is that “We’ll calculate the distance (d) in such a way that no positive or negative point can cross the margin line”. Let’s write these constraints math $y_i(\vec{w} \cdot \vec{X} + b) \geq 1$
- Suppose a green point is correctly classified that means it will follow $\mathbf{w} \cdot \mathbf{x} + \mathbf{b} \geq 1$, if we multiply this with $\mathbf{y}=1$ we get this same equation mentioned above. Similarly, if we do this with a red point with $\mathbf{y}=-1$ we will again get this equation. Hence, we can say that we need to maximize (d) such that this constraint holds true.





- We will select 2 support vectors:
 - 1 from the negative class.
 - 1 from the positive class.
- The distance between these two vectors, x_1 and x_2
- Our goal is to find the shortest distance between these two points.
- This can be achieved using a trick from the dot product:
 - Take a vector w that is perpendicular to the hyperplane.
 - Find the projection of the vector $(x_2 - x_1)$ onto w .
- **Note:** *this perpendicular vector should be a unit vector then only this will work. Why this should be a unit vector? This has been explained in the dot-product section. To make this 'w' a unit vector we divide this with the norm of 'w'.*
-

Finding Projection of a Vector on Another Vector Using Dot Product

- We already know how to find the projection of a vector on another vector. We do this by dot-product of both vectors. So let's see how

-

$$\begin{aligned} D &\Rightarrow (x_2 - x_1) \cdot \frac{\vec{w}}{\|\vec{w}\|} \\ &\Rightarrow \frac{x_2 \cdot \vec{w} - x_1 \cdot \vec{w}}{\|\vec{w}\|} \quad \text{--- (1)} \end{aligned}$$



- Since x_2 and x_1 are support vectors and they lie on the hyperplane, hence they will follow $y_i^* (\vec{w} \cdot x + b) = 1$ so we can write it as:

for positive point $y = 1$

$$\Rightarrow 1 \times (\vec{w} \cdot x_1 + b) = 1$$

$$\Rightarrow \vec{w} \cdot x_1 = 1 - b \quad \text{--- (2)}$$

Similarly for negative point $y = -1$

$$\Rightarrow -1 \times (\vec{w} \cdot x_2 + b) = 1$$

$$\Rightarrow \vec{w} \cdot x_2 = -b - 1 \quad \text{--- (3)}$$

- Putting equations (2) and (3) in equation (1) we get:

$$\Rightarrow \frac{(1 - b) - (-b - 1)}{\|\vec{w}\|}$$

$$\Rightarrow \frac{1 - b + b + 1}{\|\vec{w}\|} = \frac{2}{\|\vec{w}\|} = d$$

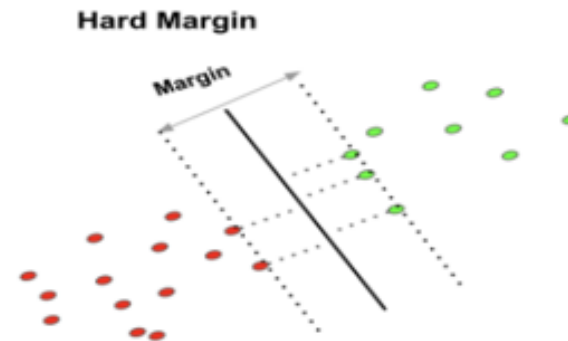
Hence the equation which we have to maximize is:

$$\operatorname{argmax}(\vec{w}^*, b^*) \frac{2}{\|\vec{w}\|} \text{ such that } y_i (\vec{w} \cdot \vec{X} + b) \geq 1$$

Hard Margin

- **Hard Margin:** If the training data is perfectly linearly separable, we can select two parallel hyperplanes that separate the two classes of data, so that the distance between them is as large as possible. So, **Hard Margin** is a maximum-margin hyperplane that perfectly separates the data without misclassifications.

Dr. Ritu



Advantages and Disadvantages of Hard Margin

- **Advantages of Hard Margin**

- **Guaranteed Separation:** Hard margin SVM ensures that the classes are perfectly separated, leading to optimal generalization performance when the training data is linearly separable.
- **Simplicity:** The optimization problem in hard margin SVM is well-defined and has a unique solution, making it computationally efficient.

- **Disadvantages of Hard Margin**

- **Sensitivity to Outliers:** Hard margin SVM is highly sensitive to outliers or noisy data points. Even a single mislabeled point can significantly affect the position of the decision boundary and lead to poor generalization on unseen data.
- **Not Suitable for Non-linear Data:** When the data is not linearly separable, hard margin SVM fails to find a valid solution, rendering it impractical for many real-world datasets.



Issue with Hard Margin Classification

- If we strictly impose that all instances be off the street and on the right side, this is called *hard margin classification*. There are two main **issues with hard margin classification**:
 - *First, it only works if the data is linearly separable, and*
 - *second it is quite sensitive to outliers.*
- Figure 5-3 shows the iris dataset with just one additional outlier: on the left, it is impossible to find a hard margin,
- In the RHS figure, the decision boundary ends up very different from the one we saw in Figure 5-1 without the outlier, and it will probably not generalize as well.

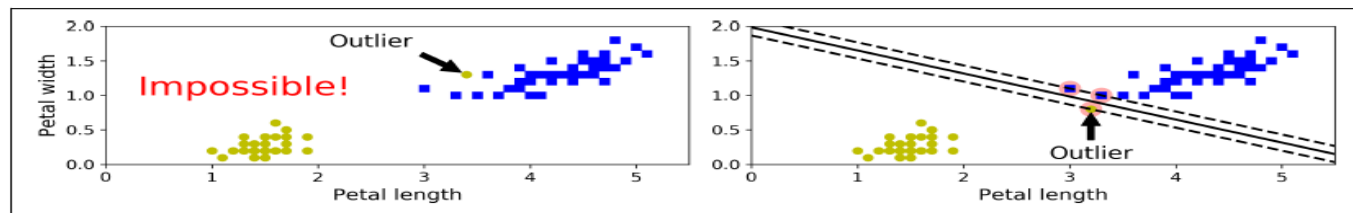


Figure 5-3. Hard margin sensitivity to outliers

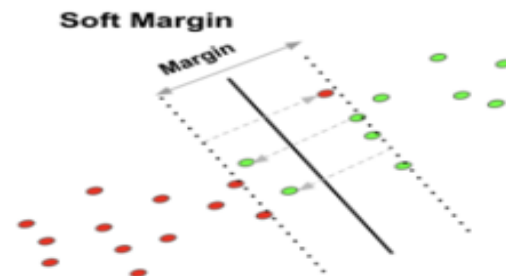
Soft Margin Classification

- To avoid the issues with Hard margin, it is preferable to use a more flexible model.
- The objective is to find a good balance between keeping the margin as large as possible and limiting the *margin violations* (i.e., instances that end up in the middle of the margin or even on the wrong side). This is called *soft margin classification*.
- In Scikit-Learn's SVM classes, you can control this balance using the **C hyperparameter**:

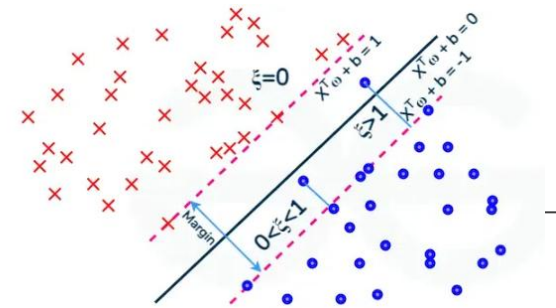


Soft Margin Classification

- **Soft Margin:** As most of the real-world data are not perfectly linearly separable, there is hardly any case we get perfectly separable data and hence we fail to use hard margin SVM. The data may contain noise or outliers. So, we will allow some margin violation to occur, which is called soft margin classification. It is better to have a large margin, even though some constraints are violated. In this way, Soft Margin SVM introduces flexibility. It introduces a penalty term for misclassifications, allowing for a trade-off between a wider margin and a few misclassifications.
- The objective is to find a good balance between keeping the margin as large as possible and limiting the *margin violations* (i.e., instances that end up in the middle of the margin or even on the wrong side). This is called *soft margin classification*.



Soft Margin SV Classifier



- **Margin violation** means choosing a hyperplane, which can allow some data points to stay in either the incorrect side of the hyperplane and between the margin and the correct side of the hyperplane. Soft Margin allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.
- Soft margin SVM allows for some margin violations. This adaptability is managed by a factor called **C**, also called the "**regularization parameter**," which helps find a balance between making the margin as big as possible and reducing mistakes in grouping things.
- In the image, we can observe some data points falling within the margin and also on the wrong side of the hyperplane, these violations are permissible in a soft margin setting. Despite these violations, the data points situated on the margins or transgressing them remain pivotal and are typically identified as support vectors.

C hyperparameter in Soft Margin Classification

- ✓ a smaller C value leads to a wider street but more margin violations.
- On the right, using a low C value the margin is quite large, but many instances end up on the street.
- ✓ On the left, using a high C value the classifier makes fewer margin violations but ends up with a smaller margin.
- ✓ The first classifier with lower c value will generalize better: in fact even on this training set it makes fewer prediction errors, since most of the margin violations are actually on the correct side of the decision boundary.
- ✓ If your SVM model is overfitting, you can try regularizing it by reducing C.

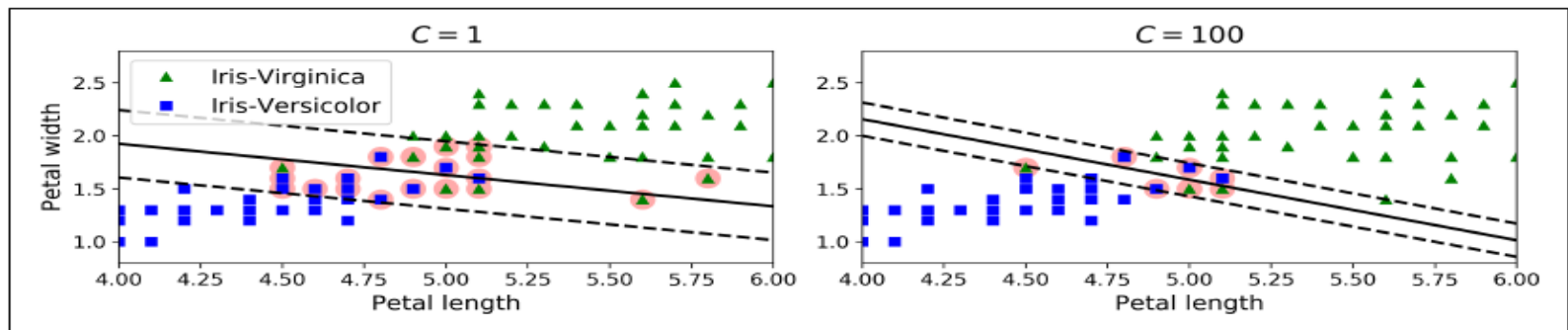


Figure 5-4. Large margin (left) versus fewer margin violations (right)

Figure 5-4 shows the decision boundaries and margins of two soft margin SVM classifiers on a nonlinearly separable dataset.

Soft margin SVM

Mathematically, decision function of a soft margin SVM can be defined as:

$$y_i(x^T \omega_i + b) \geq 1 - \xi_i$$

Where:

- ξ_i are slack variables representing the margin violations.
- y_i are the target labels.
- The term $(1 - \xi_i)$ represents the minimum required margin for each data point.

The equation introduces slack variables (ξ_i) to allow for some margin violations (misclassifications). The term $(1 - \xi_i)$ represents the minimum required margin for each data point. If a point falls within the margin or on the wrong side, its corresponding value becomes positive, reflecting the amount of violation.



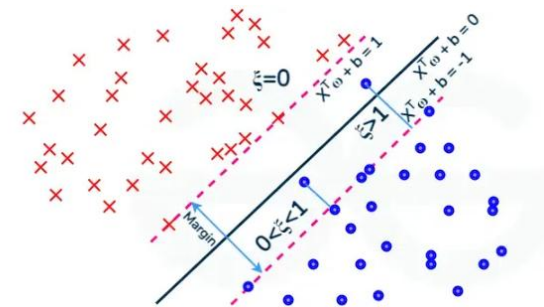
Soft Margin

We know that $\max[f(x)]$ can also be written as $\min[1/f(x)]$, it is common practice to minimize a cost function for optimization problems; therefore, we can invert the function.

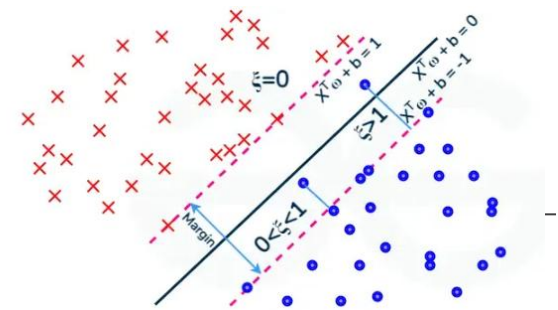
$$\operatorname{argmin}(w^*, b^*) \frac{\|w\|}{2} \text{ such that } y_i(\vec{w} \cdot \vec{X} + b) \geq 1$$

To make a soft margin equation we add 2 more terms to this equation which is **zeta** and multiply that by a **hyperparameter 'c'**

$$\operatorname{argmin}(w^*, b^*) \frac{\|w\|}{2} + c \sum_{i=1}^n \zeta_i$$



Value of Slack variable



In a Soft-Margin SVM, the slack variable ζ_i measures the degree to which a point x_i violates the margin $y_i(w^T x_i + b) \geq 1$.

- **Correctly Classified (Outside Margin): $\zeta = 0$.**

If a point is on the correct side of its respective margin hyperplane (the "L1" or "L2" boundaries you mentioned), it satisfies the hard constraint and requires no slack.

- **Correctly Classified (Inside Margin): $0 < \zeta \leq 1$.**

If a point is on the correct side of the central decision hyperplane but has crossed its "margin" hyperplane (L1 or L2), it is technically correctly classified but still has a positive ζ value.

- **Incorrectly Classified: $\zeta > 1$.**

If a point crosses the central decision hyperplane to the wrong side, ζ becomes greater than 1. Its value represents the distance from the point to its **correct margin hyperplane**, not just the central decision boundary.

3. Summary Table

Point Location	Classification Status	ζ Value
Outside Margin (Correct side)	Correct	$\zeta = 0$
On the Margin Hyperplane	Correct	$\zeta = 0$
Between Margin & Decision Line	Correct	$0 < \zeta < 1$
On the Decision Hyperplane	Uncertain	$\zeta = 1$
Beyond Decision Line (Wrong side)	Incorrect	$\zeta > 1$



Advantages and Disadvantages of Soft Margin SVM

- **Advantages of Soft Margin SVM**

- **Robustness to Outliers:** Soft margin SVM can handle outliers or noisy data more effectively by allowing for some misclassifications. This results in a more robust decision boundary that generalizes better to unseen data.
- **Applicability to Non-linear Data:** Unlike hard margin SVM, soft margin SVM can handle non-linearly separable data by implicitly mapping it to a higher-dimensional space using kernel functions. This enables SVM to capture complex decision boundaries.

- **Disadvantages of Soft Margin SVM**

- **Need for Parameter Tuning:** The performance of soft margin SVM heavily depends on the choice of the regularization parameter C . Selecting an appropriate value for C requires careful tuning, which can be time-consuming and computationally expensive, especially for large datasets.
- **Potential Overfitting:** A high value places a heavy penalty on misclassifications, forcing the SVM to create a narrow margin to classify almost all training points correctly. This acts more like a hard-margin SVM, making it highly sensitive to outliers and leading to a low-bias, high-variance model that is prone to overfitting.



Hard Margin vs Soft Margin in SVM

Criteria	Hard Margin	Soft Margin
Objective Function	Maximize margin.	Maximize margin, minimize margin violations.
Handling Noise	Sensitive, requires perfectly linearly separable data	Robust, handles noisy data with margin violations.
Regularization	Not applicable, no regularization parameter	Controlled by regularization parameter C.
Complexity	Simple, computationally efficient	May require more computational resources



SVMs are sensitive to the feature scales

- SVMs are sensitive to the feature scales, as you can see in Figure 5-2: on the left plot, the vertical scale is much larger than the horizontal scale, so the widest possible street is close to horizontal.
- After feature scaling (e.g., using Scikit-Learn's StandardScaler), the decision boundary looks much better (on the right plot).

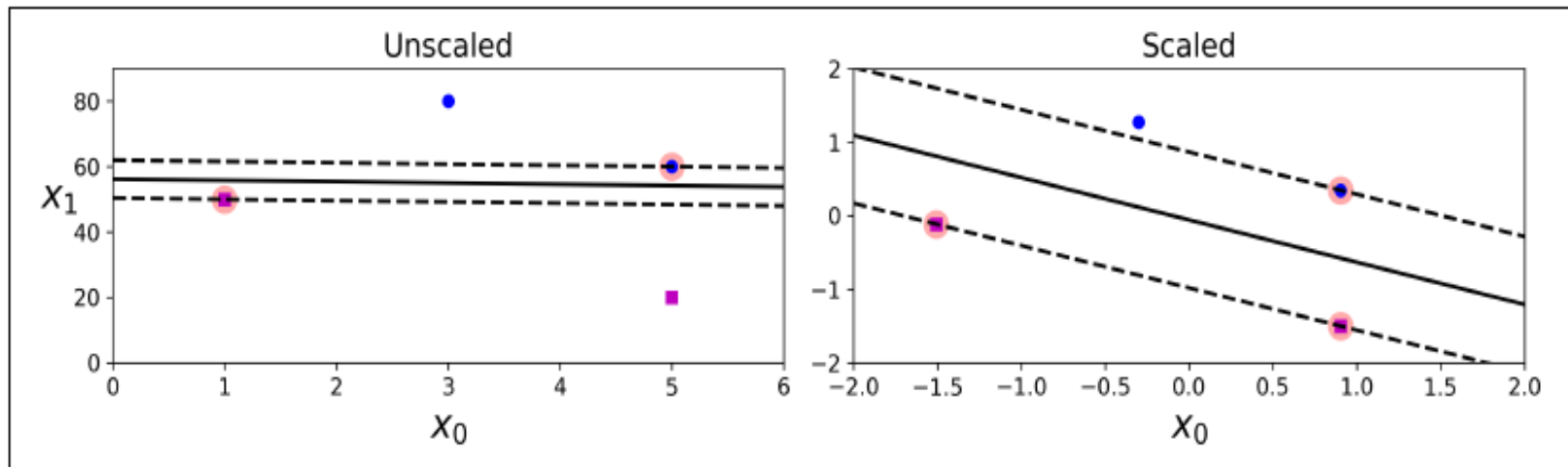


Figure 5-2. Sensitivity to feature scales

Types of Support Vector Machine

- Based on the nature of the decision boundary, Support Vector Machines (SVM) can be divided into two main parts:
- **Linear SVM:** Linear SVMs use a linear decision boundary to separate the data points of different classes. When the data can be precisely linearly separated, linear SVMs are very suitable. This means that a single straight line (in 2D) or a hyperplane (in higher dimensions) can entirely divide the data points into their respective classes. A hyperplane that maximizes the margin between the classes is the decision boundary.
- **Non-Linear SVM:** Non-Linear SVM can be used to classify data when it cannot be separated into two classes by a straight line (in the case of 2D). By using kernel functions, nonlinear SVMs can handle nonlinearly separable data. The original input data is transformed by these kernel functions into a higher-dimensional feature space where the data points can be linearly separated. A linear SVM is used to locate a nonlinear decision boundary in this modified space.



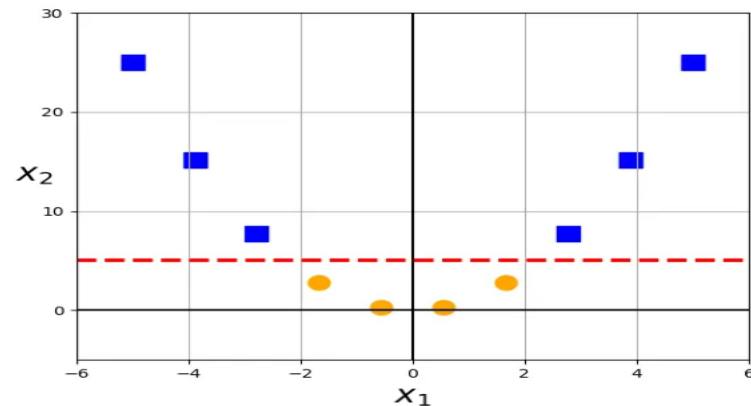
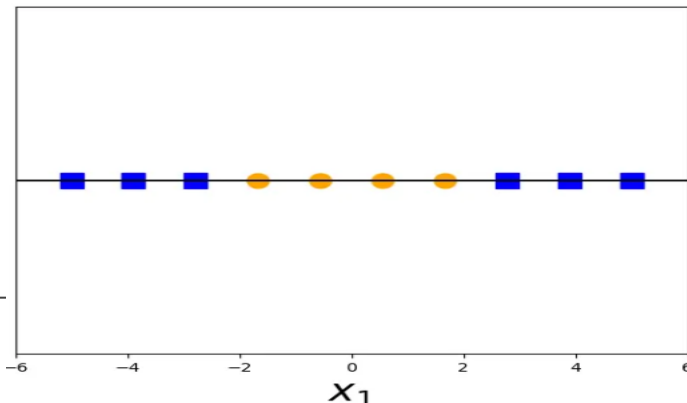
Non-linear transformations

- If the data is not linearly separable in the original, or input, space then we apply transformations to the data, which map the data from the original space into a higher dimensional feature space.
- The goal is that after the transformation to the higher dimensional space, the classes are now linearly separable *in this higher dimensional feature space*.
- We can then fit a decision boundary to separate the classes and make predictions. The decision boundary will be a hyperplane in this higher dimensional space.



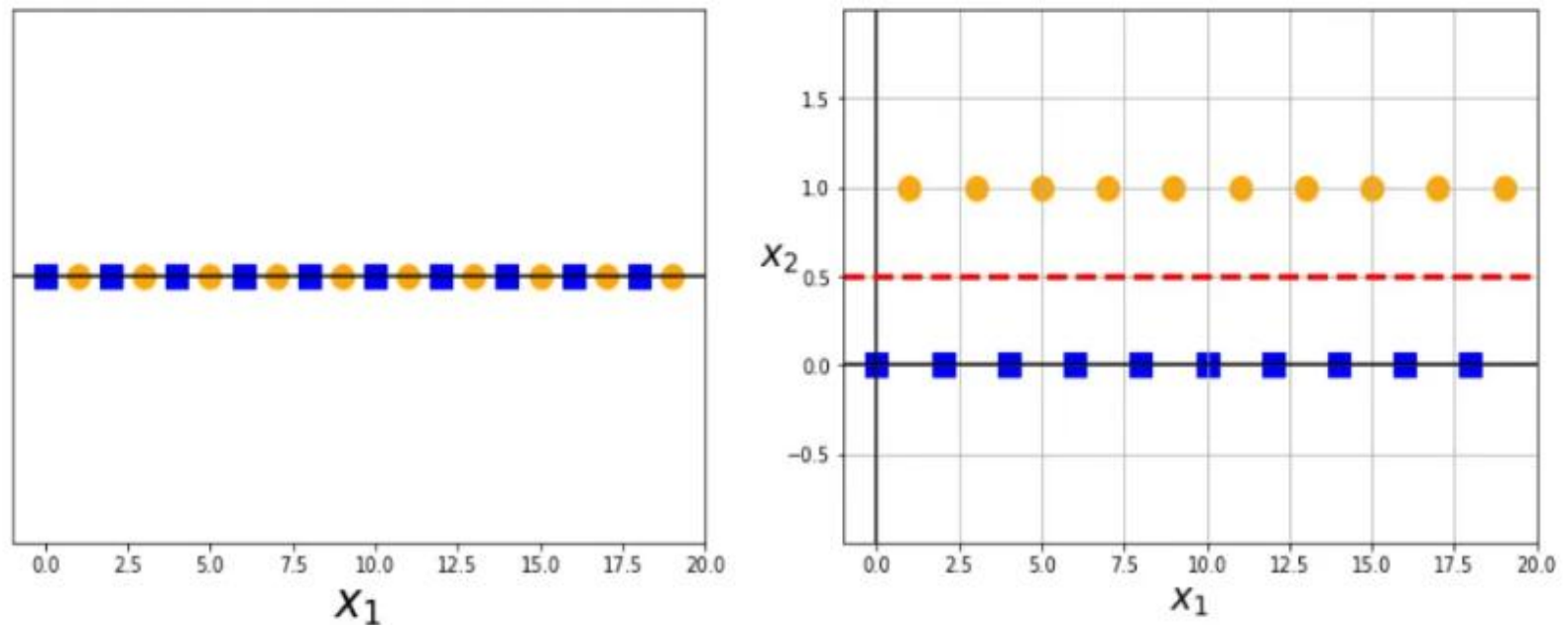
Non Linear SVM

- In this example, the picture on the left shows our original data points. In 1-dimension, this data is not linearly separable, but after applying the transformation $\phi(x) = x^2$ and adding this second dimension to our feature space, the classes become linearly separable.
- For now, we are just examining transformations of the original data to higher dimensions that allow the data to be linearly separated. These are just functions, and there are many possible functions that can map the data to any number of higher dimensions.



Non Linear SVM

Here we apply the transformation $\phi(x) = x \bmod 2$



This transformation allows us to linearly separate the even and odd x_1 values in 2 dimensions.

Non Linear SVM

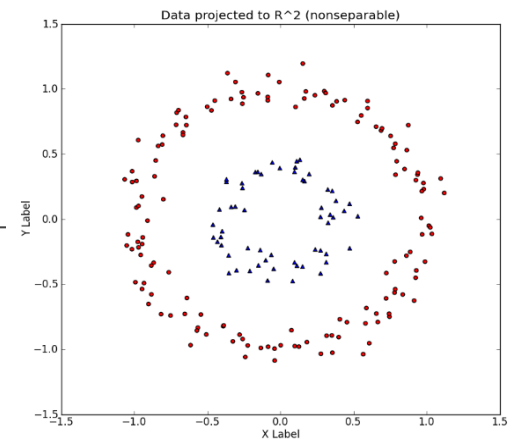
- Sometimes data is **not linearly separable** in its original space.
 - Ex: Points arranged in a **circle**
 - Inner points → Class A
 - Outer points → Class B
 - You **cannot draw a straight line** to separate them
- **The idea behind the kernel trick**
 - Instead of separating in the original space, we:
 - **Map data to a higher-dimensional space where it becomes linearly separable**



Kernel Trick: Step-by-step intuition

- **Step 1: Original 2D space**

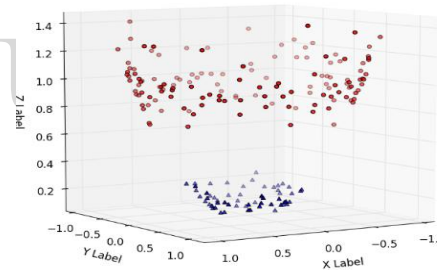
- You have $x = (x_1, x_2)$
- Data is not separable with a line



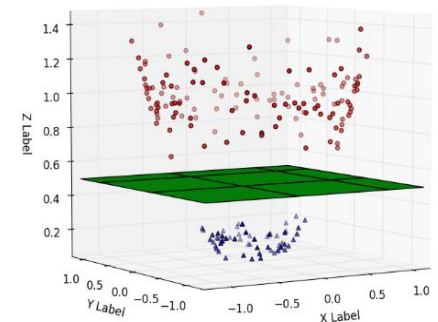
- **Step 2: Transform to higher dimension**

- Define a transformation: $\phi(x) = (x_1, x_2, x_1^2 + x_2^2)$
- Now data moves into **3D**

Data in R^3 (separable)



Data in R^3 (separable w/ hyperplane)



- **Step 3: Magic happens**

- In this new space:
- Inner circle points $\rightarrow$ small $x_1^2 + x_2^2$
- Outer circle points $\rightarrow$ large $x_1^2 + x_2^2$
- Now you can separate using a **plane**. So, Linear separation

Where is the “trick”?

- Normally, this mapping:
 - Is **computationally expensive**
 - Especially in very high dimensions
- The **Kernel Trick** is:
 - We never actually compute the transformation $\phi(x)$
 - SVM only needs **dot products** between points:
 - $\phi(x_i) \cdot \phi(x_j)$
- Instead of computing $\phi(x)$, we directly compute:
 - $K(x_i, x_j)$
 - This is called a **kernel function**



Kernel Function in SVM

- Kernel Function is a method used to take data as input and transform it into the required form of processing data.
- It computes how similar two points look after being projected into a higher feature space, without ever performing the projection.

Dr. Ritu Jain



Non Linear SVM: The Kernel Trick

- We have seen how higher dimensional transformations can allow us to separate data in order to make classification predictions.
- It seems that in order to train a support vector classifier and optimize our objective function, we would have to perform operations with the higher dimensional vectors in the transformed feature space.
- In real applications, there might be many features in the data and applying transformations that involve many polynomial combinations of these features will lead to extremely high and impractical computational costs.



Non Linear SVM: The Kernel Trick

- There can be many transformations that allow the data to be linearly separated in higher dimensions, but not all of these functions are actually kernels.
- The kernel function has a special property that makes it particularly useful in training support vector models, and the use of this property in optimizing non-linear support vector classifiers is often called the **kernel trick**.

Dr. Ritu Jain



Non Linear SVM: The Kernel Trick

- The **kernel trick** provides a solution to this problem. The “trick” is that kernel methods represent the data only through a set of pairwise similarity comparisons between the original data observations $\mathbf{x}$ (with the original coordinates in the lower dimensional space), instead of explicitly applying the transformations $\phi(\mathbf{x})$ and representing the data by these transformed coordinates in the higher dimensional feature space.
- In kernel methods, the data set $\mathbf{X}$ is represented by an $n \times n$ kernel matrix of pairwise similarity comparisons where the entries (i, j) are defined by the **kernel function**: $k(\mathbf{x}_i, \mathbf{x}_j)$. This kernel function has a special mathematical property. The kernel function acts as a modified dot product. We have:



Kernel Definition

- A function that takes as its inputs vectors in the original space and returns the dot product of the vectors in the feature space is called a *kernel function*
- More formally, if we have data $\mathbf{x}, \mathbf{z} \in X$ and a map $\phi : X \rightarrow \mathbb{R}^N$ then

$$k(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

is a kernel function



Non Linear SVM: Kernel Trick

- Our kernel function accepts inputs in the original lower dimensional space and returns the dot product of the transformed vectors in the higher dimensional space.
- It can somewhat help to understand how the kernel function is equal to the dot product of the transformed vectors by considering that each coordinate of the transformed vector $\phi(\mathbf{x})$ is just some function of the coordinates in the corresponding lower dimensional vector $\mathbf{x}$.



Non Linear SVM: Kernel Trick

- The ultimate benefit of the kernel trick is that the objective function we are optimizing to fit the higher dimensional decision boundary only includes the dot product of the transformed feature vectors. Therefore, we can just substitute these dot product terms with the kernel function, and we don't even use $\phi(\mathbf{x})$.
- Remember, our data is only linearly separable as the vectors $\phi(\mathbf{x})$ in the higher dimensional space, and we are finding the optimal separating hyperplane in this higher dimensional space *without having to calculate or in reality even know anything about $\phi(x)$.*



Popular Kernel functions

The choice of kernel determines the shape of the decision boundary.

- **Linear Kernel:** $K(x, y) = x \cdot y$. Used when data is already linearly separable or for high-dimensional sparse data like text.
- **Polynomial Kernel:** $K(x, y) = (x \cdot y + c)^d$. Useful for capturing interactions between features. The degree (d) controls the complexity of the boundary.
- **Radial Basis Function (RBF) / Gaussian Kernel:** $K(x, y) = \exp(-\gamma ||x - y||^2)$. The most versatile and common choice. It can map data into an infinite-dimensional space and is effective for complex, non-linear relationships.
- **Sigmoid Kernel:** $K(x, y) = \tanh(\alpha x \cdot y + \beta)$. Primarily used as a proxy for neural networks, mimicking activation function behavior.



Mathematical Formula for kernel function

- Mathematically, SVM Kernel Function can be represented by:

$$K(x, x') = \phi(x)^\top \phi(x')$$

1. x, x' : Two feature vectors
2. $\phi(\cdot)$: Mapping function to higher-dimensional space
3. $\phi(x)^\top \phi(x')$: Dot product measuring similarity after mapping
4. $K(x, x')$: Kernel similarity score

Instead of computing $\phi(x)$ explicitly which can be expensive we compute the similarity directly using K , which is faster. This is called the kernel trick.



Linear Kernel

- Measures similarity using a simple dot product; best for linearly separable data with fast computation

Formula

$$K(x, x') = x^\top x'$$

Where,

- x, x' : Feature vectors
- $x^\top x'$: Similarity between the two points



Characteristics of Linear kernel

- **Dot Product Similarity:** It evaluates similarity using a straightforward dot product between feature vectors.
- **Straight Boundaries:** It works best when the data is linearly separable.
- **Fast Training:** It trains faster compared to more complex kernels.
- **Sparse Features:** It is commonly used for text or document classification tasks.
- **Wide Margins:** It produces boundaries that remain clean and interpretable.



RBF (Gaussian) Kernel

- Maps data into infinite dimensions and works well when class boundaries are highly non-linear and irregular.

Formula

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Where,

1. *exp*: Exponential curve function
2. γ : Controls how far influence of a point spreads
3. $\|x - x'\|^2$: Squared distance between points



Radial Basis Function (RBF) kernel

- The Radial Basis Function (RBF) kernel is the default and most popular kernel used in Support Vector Machines (SVM) because of its ability to handle complex, non-linear data distributions.
- It maps input data into an **infinite-dimensional** feature space, allowing it to find a separating hyperplane that is impossible to achieve in lower-dimensional space.

Dr. Ritu Jain



Primary utilities of the RBF kernel

- **Handling Non-Linearity:** It allows SVMs to create complex, curved decision boundaries for datasets where classes cannot be separated by a straight line.
- **Infinite Dimensional Mapping:** Through the "kernel trick," the RBF kernel implicitly projects data into a space of infinite dimensions without needing to compute the actual coordinates, making it more powerful than finite kernels like the polynomial kernel.
- **Similarity-Based Classification:** It functions as a similarity measure between data points. Two points are considered similar if they are close in the input space; the kernel value is 1 for identical points and decays toward 0 as distance increases.



Primary utilities of the RBF kernel

- **Local Influence:** It behaves like a weighted nearest-neighbor model. Only nearby support vectors significantly influence the classification of a new point, capturing local structures in the data.
- **Computational Efficiency:** Despite its "infinite" flexibility, it is computationally efficient because it only requires calculating Euclidean distances between points rather than complex feature expansions.

Dr. Ritu Jain



Gamma in RBF kernel function

- In the Radial Basis Function (RBF) kernel of a Support Vector Machine (SVM), gamma is a hyperparameter that defines the radius of influence of individual training points.
- Intuitively, the gamma parameter defines how far the influence of a single training example reaches, with low values meaning 'far' and high values meaning 'close'.
- The gamma parameters can be seen as the inverse of the radius of influence of samples selected by the model as support vectors.
- Low gamma means a far-reaching influence (smooth boundary), while high gamma causes a localized, "wiggly" boundary, which can lead to overfitting by capturing noise.
- Low gamma: High bias, low variance (simpler model).
- High gamma: Low bias, high variance (complex model).
- Typical Ranges: Often tested in ranges like 0.0001 to 10



Gamma in RBF kernel function

- Gamma determines how far the influence of a single training point reaches.
- **Kernel function for RBF:**
- **Low gamma ($\gamma \downarrow$)**
 - The exponential decays **slowly**
 - Each data point influences a **large region**
 - Decision boundary becomes **smooth and simple**
 - Effect:
 - High bias (model too simple $\rightarrow$ may underfit)
 - Low variance (stable, less sensitive to noise)
 - Intuition:
“Far-away points still look similar.”

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$



Gamma in RBF kernel function

- **High gamma ($\gamma \uparrow$)**
 - The exponential decays **very quickly**
 - Each point influences only a **small neighborhood**
 - Decision boundary becomes **highly wiggly and complex**
 - Effect:
 - Low bias (fits training data very closely)
 - High variance (overfits, sensitive to noise)
 - Intuition:
“Only very close points matter.”



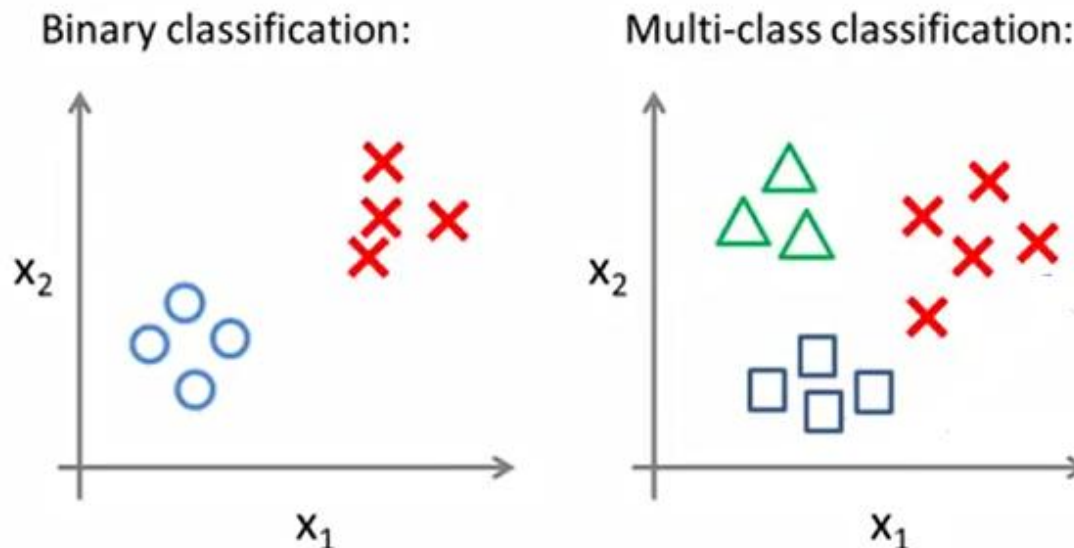
Key Tuning Parameters of RBF

- The utility of the RBF kernel depends heavily on two hyperparameters:
- **Gamma** : Controls the reach of a single training example. A high gamma makes the influence of a point very localized (risking overfitting), while a low gamma makes the influence broader, creating smoother decision boundaries.
- **C**: The regularization parameter that balances the trade-off between maximizing the margin and minimizing classification errors. High values of C prioritize correct classification of training data, while low values prioritize a simpler, smoother decision surface



Multi-class Classification

- Multiclass classification is a supervised machine learning task that assigns a single, mutually exclusive label to an instance from three or more distinct categories (e.g., classifying images as cat, dog, or bird).



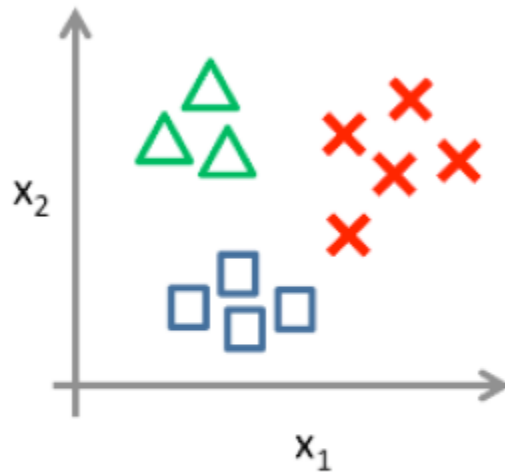
Strategies for Multi-class Classification with SVM

- When dealing with multi-class classification using Support Vector Machines (SVM), two primary strategies are commonly employed:
 - **One-vs-One (OvO)** and
 - **One-vs-All (OvA)**.
- Each approach has its own methodology and application scenarios, making them suitable for different types of classification problems.
- One-vs-All (OvA/OvR)—training one classifier per class against all others—or
- One-vs-One (OvO)—training a classifier for every possible pair to handle multi-class problems.
- OvA creates N classifiers, while OvO creates $N(N-1)/2$ classifiers

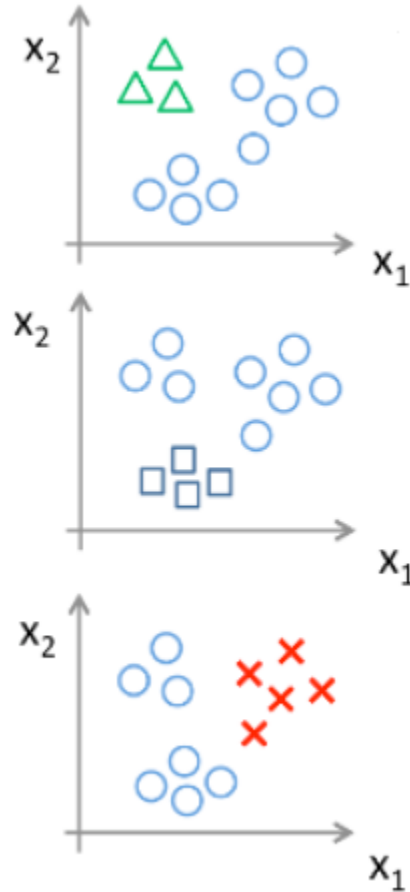


Multi-class Classification (One-vs-all)

One-vs-all (one-vs-rest):



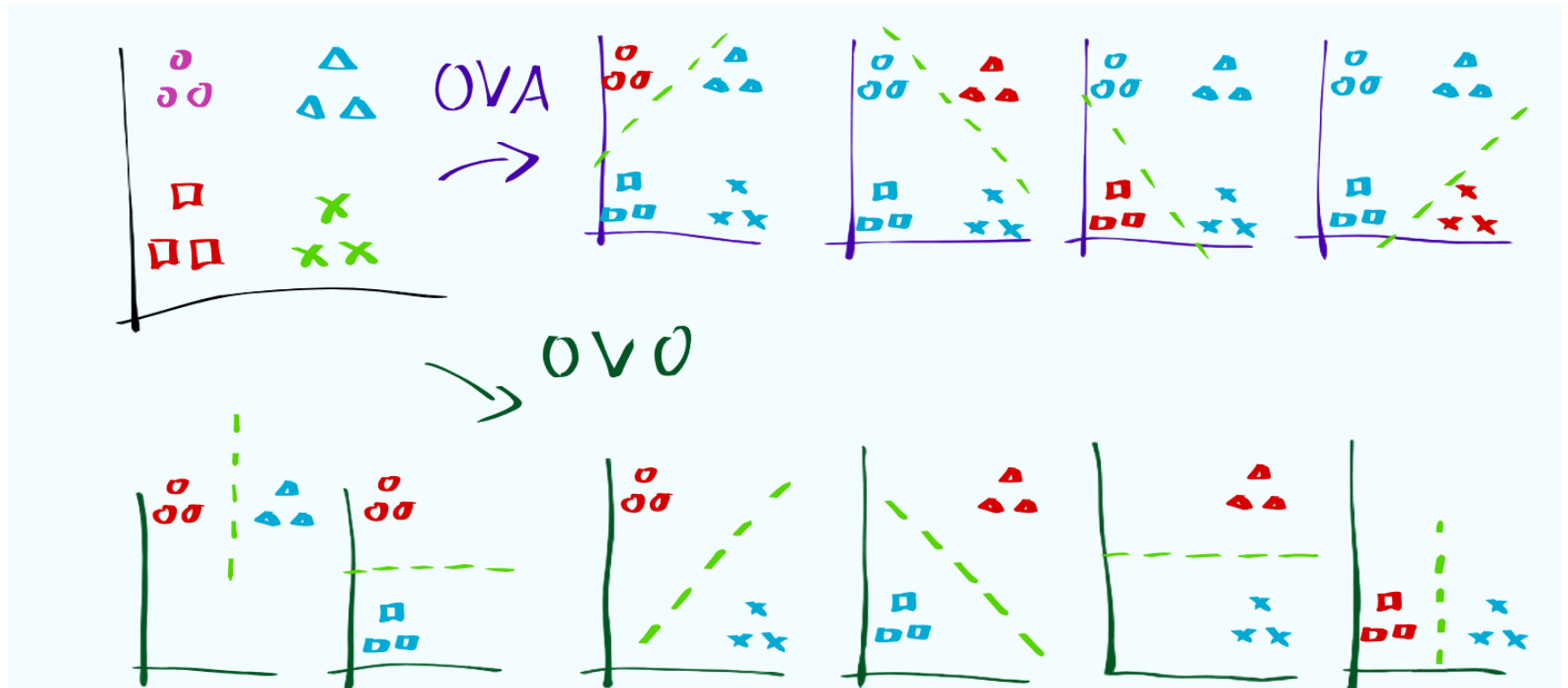
Class 1: 
Class 2: 
Class 3: 



The following image shows how one could classify 3 classes:



OVA and OVO



One-vs-All (OvA) Approach

- The One-vs-All (OvA) approach, also known as **One-vs-Rest**, involves training a single binary classifier for each class.
- In this method, each class is treated as the positive class, while all other classes are grouped together as the negative class.
- This approach is straightforward and scales linearly with the number of classes.

Dr. Ritu Jain



Implementation:

One-vs-All (OvA) Approach

- **Training Phase:**

- For each class, a binary SVM classifier is trained. For a dataset with n classes, this results in n classifiers.
- Each classifier is trained to distinguish its respective class from all other classes combined.

- **Prediction Phase:**

- During prediction, each classifier outputs a score or probability indicating the likelihood of the instance belonging to its respective class.
- The class with the highest score is selected as the prediction for the instance.



One-vs-One (OvO) Approach

- The One-vs-One (OvO) approach involves creating a binary classifier for every possible pair of classes.
- This method is particularly advantageous when the number of classes is relatively small, as it allows for a focused comparison between pairs of classes.
- Each classifier is responsible for distinguishing between two specific classes, effectively ignoring the others.



Implementation of OvO

- **Training Phase:**
 - For each pair of classes, a binary SVM classifier is trained. For instance, if there are three classes (A, B, and C), the OvO approach will train classifiers for (A vs B), (A vs C), and (B vs C).
 - Each classifier is trained using only the data points belonging to the two classes it is meant to distinguish.



Implementation of OvO

- **Prediction Phase:**

- During prediction, each binary classifier provides a vote for one of the two classes it was trained on.
- The final class prediction for a given instance is determined by a majority vote among all the classifiers.
- This voting mechanism ensures that the class with the most votes is selected as the final output.



Choosing Between OvO and OvA

- The choice between One-vs-One and One-vs-All strategies depends on several factors, including the number of classes, the size of the dataset, computational resources, and the specific requirements of the application.
 - **One-vs-One** is generally preferred when the number of classes is small and computational resources are sufficient, as it can provide higher accuracy through focused pairwise comparisons.
 - **One-vs-All** is more suitable for scenarios with a large number of classes or when simplicity and computational efficiency are prioritized.